Online Appendices for *dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Processors*

Sanjiang Li
Centre for Quantum Software and Information, University of Technology Sydney, Australia
Email: sanjiang.li@uts.edu.au

This document collects the online appendices referenced from the main paper. Cross-references with prefix `sec:` and `tab:` (e.g. Section III-C of the main paper) refer to the main manuscript and are not resolved here; section and table numbering in this document is self-contained.

Guide to the appendices. The appendices supplement the main paper with material that did not fit the page limit but is useful for reproducibility and for readers interested in specific design or empirical questions. Appendix A ablates two scoring components (node decay, hop gain) in detail. Appendix B reports an architecture-sensitivity study on inter-core link density. Appendix C compares initial-layout pipelines (TELESABRE layout, random, and SabreLayout). Appendices D and E provide the detailed code-level audits, head-to-head tables, and reproduction instructions for the `pytket-dqc` and DMapS comparisons summarised in the main text.

APPENDIX A
COMPONENT ABLATIONS

This appendix supplements the mechanism-ablation table in Section IV-C of the main paper with per-circuit results for two scoring components: the node-decay anti-oscillation guard inherited from SABRE, and the inter-core hop-gain reward g_{hop}.

A. Node-Decay

DSABRE inherits from SABRE a *node-decay* penalty $\delta : \mathrm{p}_p \to \mathbb{R}_{\geq 1}$ incremented by 0.1 on every SWAP touching p and reset to 1 when a gate executes on p or a teleportation departs from a comm port. It multiplies the SWAP score (Eq. 3) and discourages two-qubit oscillation. We ablate the term by toggling `enable_node_decay` under the headline protocol (corners-removed best-of-3 SabreLayout, fwd→bwd→fwd schedule).

EPR results. Node decay has a circuit-dependent and scale-dependent effect on EPR count. At 25q the impact is negligible: five of six circuits tie exactly and the geometric-mean advantage of decay ON is only $+0.9\%$. At 36q the picture favours decay ON more strongly ($+8.3\%$ gmean), driven primarily by `vqe_su2`, where decay prevents oscillation on the two high-traffic qubit slots that carry almost all inter-core traffic; without the penalty the router repeatedly re-selects the same two SWAPs and accumulates six extra EPR pairs ($9 \to 15$). At 64q the balance reverses: decay ON *increases* EPR by -2.7% gmean, with `graphstate` as the largest outlier (-21.1%). On the larger H-grid topology, anti-oscillation pressure forces the router onto longer SWAP detours that push qubits into congested core boundaries and trigger additional teleportations.

TABLE I
NODE-DECAY ABLATION ON THE 25Q SUITE (B-GRID 2×2, 4×4, 25-QUBIT CIRCUITS). ΔEPR% > 0 MEANS DECAY ON IS BETTER.

Circuit	EPR (on)	EPR (off)	ΔEPR%	LS (on)	LS (off)
ae	23	23	0.0	374	304
ghz	1	1	0.0	14	14
graphstate	2	2	0.0	16	14
qft	33	33	0.0	461	349
qnn	48	48	0.0	1076	701
random	169	178	+5.3	1374	1123
gmean	**15.2**	**15.3**	**+0.9**		

TABLE II
NODE-DECAY ABLATION ON THE 36Q SUITE (B-GRID 2×2, 4×4, 36-QUBIT CIRCUITS).

Circuit	EPR (on)	EPR (off)	ΔEPR%	LS (on)	LS (off)
bv	1	1	0.0	8	9
dj	3	3	0.0	24	22
qaoa	145	140	−3.4	1319	1006
qpeexact	65	65	0.0	874	608
vqe_su2	9	15	+66.7	52	59
wstate	8	8	0.0	63	36
gmean	**11.3**	**12.2**	**+8.3**		

Local-SWAP results. Across all three suites and all circuits, decay OFF uses markedly fewer local SWAPs—reductions range from $\approx 15\%$ to $\approx 40\%$. When the router is free to reuse the same efficient SWAP path, it reaches executable adjacency with fewer moves. Decay ON deflects the router onto alternative, often longer, paths that happen to avoid recently penalised qubits.

Takeaway. Node decay is a useful anti-oscillation guard on small, structured circuits where a handful of qubits dominate inter-core traffic (36q, $+8.3\%$ EPR). At larger scale (64q), the forced diversification of SWAP paths increases congestion

TABLE III
NODE-DECAY ABLATION ON THE 64Q SUITE (H-GRID 2×3, 4×4, 64-QUBIT CIRCUITS).

Circuit	EPR (on)	EPR (off)	ΔEPR%	LS (on)	LS (off)
ae	216	203	−6.0	2332	1363
ghz	16	16	0.0	89	72
graphstate	19	15	−21.1	104	95
qft	246	285	+15.9	2054	1742
qnn	521	504	−3.3	8662	5167
random	714	730	+2.2	3814	3641
gmean	**134.8**	**131.2**	**−2.7**		

and net EPR cost (-2.7%). The mechanism is retained in the default configuration because the 25q/36q suites represent the target regime of current multi-core hardware, and the 64q loss is modest. Future work could make the decay step-size a tunable hyperparameter or apply it only when the front layer contains a persistent inter-core bottleneck qubit.

B. Hop-Gain

The hop-gain term $g_{\text{hop}} = w_h\,(d_{\text{core}}(c_{\text{src}}, c_{\text{tgt}}) - d_{\text{core}}(c_{\text{next}}, c_{\text{tgt}}))$ in Eq. 4 adds a position-independent directional reward on top of the front-layer gain Δ_F. Because inter-core links carry weight $w_{\text{link}} \gg 1$, Δ_F already captures core-level direction; g_{hop} compensates when the specific landing port within c_{next} is far from the next staging position, depressing Δ_F below the true directional value. We ablate g_{hop} by zeroing it (`enable_hop_gain=False`) under the same protocol as Appendix A-A; the term is a single $O(1)$ lookup so compile time is unaffected.

TABLE IV
HOP-GAIN ABLATION ACROSS ALL THREE BENCHMARK SUITES. ΔEPR% > 0 MEANS HOP-GAIN ON IS BETTER. ALL LOCAL-SWAP COUNTS ARE IDENTICAL BETWEEN VARIANTS UNLESS NOTED.

Suite	Circuit	EPR (on)	EPR (off)	ΔEPR%
25q	ae	23	23	0.0
	ghz	1	1	0.0
	graphstate	2	2	0.0
	qft	33	33	0.0
	qnn	48	48	0.0
	random	169	169	0.0
	gmean	**15.2**	**15.2**	**0.0**
36q	bv	1	1	0.0
	dj	3	3	0.0
	qaoa	145	145	0.0
	qpeexact	65	65	0.0
	vqe_su2	9	9	0.0
	wstate	8	8	0.0
	gmean	**11.3**	**11.3**	**0.0**
64q	ae	216	216	0.0
	ghz	16	17	+6.2
	graphstate	19	19	0.0
	qft	246	246	0.0
	qnn	521	521	0.0
	random	714	751	+5.2
	gmean	**134.8**	**137.4**	**+1.9**

Results and discussion. On the B-grid suites (25q, 36q) every circuit ties exactly in both EPR count and local-SWAP count: the weighted physical distance encoded in Δ_F (inter-core link weight w_{link}=10) already carries the full directional signal, and g_{hop} adds no independent information. On the H-grid suite (64q) the term is nearly inert—four of six circuits still tie—but provides a measurable benefit on the two circuits where the routing reaches a decision boundary: `random` ($+5.2\%$) and `ghz` ($+6.2\%$), yielding a gmean improvement of $+1.9\%$. The benefit on these circuits is consistent with the landing-position argument in Section III-C: the H-grid's longer inter-core paths mean that the specific comm port at which a qubit arrives can be far from the best staging position for the next hop, depressing Δ_F below the true directional value; g_{hop} compensates with a fixed positive signal independent of landing position.

Takeaway. g_{hop} is redundant with Δ_F on the B-grid but provides a small, consistent gain on the larger H-grid topology. It is retained in the default configuration; its potential role on even larger or more deeply hierarchical architectures (where landing-position variance grows) motivates keeping it as a tunable supplement rather than removing it.

APPENDIX B
EFFECT OF INTER-CORE LINK DENSITY

We investigate whether increasing the number of quantum links in the B-grid benefits routing. In the standard 2×2 B-grid each adjacent core pair shares a single physical link (4 links total). We construct an *enhanced* variant that adds a second parallel link between every adjacent pair, yielding 8 inter-core links. The new endpoints are placed at the interior boundary positions symmetric to the originals: on a 4×4 core, where the original C_0–C_1 link uses row 1 of the boundary, the new link uses row 2 (and vice versa for C_2–C_3), and symmetrically for the vertical pairs. No corner nodes are involved in any link. Both compilers (TeleSABRE and DSABRE) and `pytket-dqc` are run on the 25q and 36q suites under the same protocol as the main evaluation (Section IV).

pytket-dqc topology model. `pytket-dqc`'s `NISQNetwork` represents inter-QPU connectivity as a simple adjacency list of core pairs, without modelling the number of physical links per pair. Its e-bit cost therefore depends only on *which* core pairs are connected, not on *how many* physical links connect them. Running `pytket-dqc` on the enhanced architecture produces identical e-bit counts (geometric means 48.1 and 16.0 for 25q and 36q respectively), confirming this invariance.

Table V reports per-circuit results. The geometric-mean change relative to the 4-link baseline is shown below each gmean row.

Key finding. Adding inter-core links does *not* reduce EPR costs; on the contrary, EPR counts increase for every compiler that models physical-link placement. The effect is mild for DSABRE (+7–14% geometric mean across suites) but pronounced for TeleSABRE on the 36q suite ($+19\%$), with `qpeexact` more than doubling ($100 \rightarrow 207$ EPRs).

Discussion. Three complementary mechanisms explain this counter-intuitive outcome.

TABLE V

EPR/E-BIT COUNTS ON STANDARD 4-LINK (4L) VS. ENHANCED 8-LINK (8L) B-GRID (2×2 4×4, 64 PHYSICAL QUBITS). PYTKET−DQC'S OUTPUT DEPENDS ONLY ON THE CORE ADJACENCY AND IS THEREFORE IDENTICAL FOR BOTH ARCHITECTURES (SINGLE COLUMN). THE Δ ROW REPORTS THE GEOMETRIC-MEAN CHANGE FROM 4L TO 8L FOR EACH COMPILER.

	pytket-dqc	TeleSABRE		dSABRE	
Circuit	e-bits	4L	8L	4L	8L
25-qubit suite					
ae	85	23	27	23	23
ghz	3	2	2	1	1
graphstate	4	11	11	2	4
qft	120	39	37	33	33
qnn	152	51	51	48	48
random	665	292	283	169	185
gmean	**48.1**	**25.8**	**26.1**	**15.2**	**17.3**
Δ (8L vs 4L)	—	+1.3%		+14%	
36-qubit suite					
bv	3	5	4	1	1
dj	3	3	5	3	3
qaoa	194	232	254	145	139
qpeexact	175	100	207	65	68
vqe_su2	9	16	15	9	12
wstate	6	12	12	8	9
gmean	**16.0**	**20.1**	**24.0**	**11.3**	**12.1**
Δ (8L vs 4L)	—	+19%		+7.0%	

(i) Communication-qubit proliferation. Each physical link requires two dedicated communication ports, one in each core. Doubling the links doubles the number of comm qubits from 8 to 16 out of 64 physical qubits. Comm qubits are attractive relay points in heuristic routing: whenever a logical qubit sits on a comm port, the router perceives an inexpensive path to the adjacent core and may initiate an otherwise unnecessary teleportation. This "teleportation temptation" grows with comm-qubit density, increasing wasteful inter-core traffic.

(ii) Layout disruption from boundary saturation. The SABRE-style layout pass places logical qubits to minimise expected routing cost. With more boundary positions occupied by comm qubits, effective placement freedom near core boundaries shrinks. Circuits whose critical qubits should remain inside a core may find their preferred positions displaced, forcing additional intra-core SWAPs or cross-core handoffs that inflate EPR counts.

(iii) Layout-heuristic miscalibration in TeleSABRE. TeleSABRE's Hungarian-method initial layout applies a fixed `inter_core_edge_weight` of 2 to penalise cross-core assignments. On the 4-link architecture this weight is well-calibrated: inter-core paths are scarce and expensive. With 8 links the effective inter-core cost in the physical graph decreases, flattening TeleSABRE's placement objective and producing noisier initial layouts. The extreme outlier `qpeexact` (36q) illustrates the hazard: a single poor initial assignment on a structured, depth-sensitive circuit cascades into more than double the teleportation count. Retuning the inter-core penalty for the denser topology would likely recover—or surpass—the 4-link baseline.

Takeaway. The standard 4-link B-grid is already well-matched to 25–36 qubit workloads on a 64-qubit device: the bottleneck is layout quality and intra-core SWAP overhead, not inter-core link bandwidth. Increasing link count without co-optimising the routing energy function and layout heuristic can substantially worsen EPR efficiency. These results suggest that future hardware design should pair link-density increases with corresponding tuning of routing parameters, rather than assuming that architectural headroom automatically translates into better compiled outcomes.

APPENDIX C
LAYOUT BASELINES

Section IV-D describes our default initial-layout pipeline (SabreLayout with corners removed, paired with a fwd/bwd/fwd routing schedule) but does not compare it against alternatives. Here we report that comparison on the 25q and 64q suites. Three layout strategies are tested:

TS TELESABRE's built-in `optimize_initial` layout.

Rand Random qubit assignment, best of three trials.

SL Qiskit `SabreLayout` on the architecture graph with the four lowest-degree corner nodes removed, best of three seeds (the default used throughout this paper).

Each layout is paired with two routing schedules: *fwd×2* (two forward passes) and *sabre* (forward → reversed-DAG backward → forward; the better of the first and third forward passes by EPR count is reported).

TABLE VI

GMEAN EPR UNDER THREE INITIAL-LAYOUT METHODS AND TWO ROUTING-PASS STRATEGIES. BOLD: BEST RESULT PER COLUMN. *Rand/SL* USE THREE RANDOM TRIALS OR SEEDS; BEST IS REPORTED. *TS reference* ROW IS TELESABRE'S END-TO-END RESULT (ITS OWN ROUTER ON ITS OWN LAYOUT) FOR CONTEXT.

Layout	Schedule	25q EPR	64q EPR
TS	fwd×2	22.5	110.3
TS	sabre	19.3	98.4
Rand	fwd×2	32.0	125.4
Rand	sabre	24.0	112.2
SL	fwd×2	17.6	112.3
SL	sabre	**15.7**	**100.2**
TS reference	—	31.1	157.6

Findings. First, *SL + sabre* is the strongest combination on 25q (15.7 gmean EPR, −30% relative to the TS + fwd×2 baseline and −49% relative to TELESABRE) and on 64q (100.2, −9%). Crucially, this combination requires no TELESABRE run—SabreLayout is a standard Qiskit pass. Second, the SABRE backward pass benefits every layout: pairing TS with the sabre schedule cuts 25q gmean from 22.5 to 19.3 (−14%) and 64q from 110.3 to 98.4 (−11%), and the corresponding Rand-layout drops are −25% and −11%. These findings support adopting SL + sabre as the default configuration: it is the best layout–schedule pair tested and requires no TELESABRE infrastructure. The slightly tighter headline gmean (15.2 on

25q, 98.4 / 100.2-region on 64q) reported in Tables IV/VI reflects the same SL+sabre combination with the corners-removal preprocessing and BFS-layer extended set tuned to current defaults.

APPENDIX D
COMPARISON WITH PYTKET-DQC

Background. `pytket-dqc` [1] is a Python library from Quantinuum that distributes circuits over multiple QPUs using *gate teleportation* (detached gates): a pre-shared e-bit allows a non-local controlled-unitary to be executed without moving the qubit, and consecutive non-local gates on the same wire can share the same e-bit (multi-gate amortisation). This contrasts with DSABRE's *state teleportation*, which physically moves a qubit into a target core so that all subsequent gates on it run locally. Both primitives consume EPR pairs, but the reported counts are not directly comparable: `pytket-dqc` charges one e-bit per edge in the Steiner tree connecting all servers in a hyperedge, with multiple consecutive non-local gates on the same wire amortised into a single hyperedge. DSABRE charges one EPR per qubit-teleportation hop regardless of how many gates follow (see Section IV-A). `pytket-dqc` assigns qubits to QPUs *statically* via KaHyPar hypergraph partitioning and does not model intra-QPU SWAP routing; its output is an e-bit count only.

Implementation notes. We install `pytket-dqc` v0.0.1 from source and `kahypar` 1.3.5. Two changes were required to run our benchmarks:

1) *KaHyPar weight-0 patch.* `pytket-dqc` represents the circuit as a hypergraph whose qubit vertices carry weight 1 and gate vertices weight 0. KaHyPar $\geq$1.3.5 rejects zero-weight vertices. We patched the allocator source file to assign weight 1 to all vertices and increase each server's target block size by $\lceil n_{\text{gates}}/k \rceil$, so the capacity constraint still effectively counts only qubit vertices.
2) *64-qubit QASM width.* The pytket QASM loader defaults to a 32-bit classical register maximum; the 64-qubit benchmarks carry a 64-bit measurement register. We pass `maxwidth=128` to `circuit_from_qasm`.

All other `pytket-dqc` internals are unmodified. We use the `HypergraphPartitioning` allocator (KaHyPar connectivity metric), best of 5 random seeds. The QPU topology matches the core graph of each architecture: a 2×2 grid (4 QPUs) for the 25q/36q B-grid and a 2×3 grid (6 QPUs) for the 64q H-grid.

Per-suite gmean results for the own-mapping comparison appear in Tables IV–VI of the main text. Here we report the shared-mapping comparison, which isolates routing quality from layout quality.

Shared-mapping comparison. To separate *routing quality* from *layout quality* we fix the initial qubit-to-core assignment to pytket-dqc's best KaHyPar partition and then run DSABRE from the same layout. Within each core the logical-to-physical slot assignment is determined by the topology-aware routine used in the main evaluation. Tables VII and VIII report per-circuit results together with wall-clock compile times.

TABLE VII
SHARED KAHYPAR MAPPING: PYTKET-DQC E-BITS VS. DSABRE EPR AND COMPILE TIME. 25Q AND 36Q SUITES, B-GRID 2×2 4×4. DSABRE USES SABRE-STYLE FWD→BWD→FWD PASSES FROM PYTKET-DQC'S BEST-SEED PARTITION.

		pytket-dqc		DSABRE		
Circuit	CX	e-bits	time	EPR	time	Δ
25q						
ae	558	85	1.0 s	23	0.9 s	−73%
ghz	24	3	0.0 s	3	0.0 s	0%
graphstate	25	4	0.0 s	5	0.0 s	+25%
qft	580	120	1.2 s	33	0.8 s	−72%
qnn	1223	152	3.7 s	49	2.7 s	−68%
random	1124	665	3.0 s	181	5.5 s	−73%
gmean		**48.1**		**21.6**		**−55%**
36q						
bv	17	3	0.0 s	1	0.0 s	−67%
dj	35	3	0.1 s	7	0.0 s	+133%
qaoa	1200	194	3.8 s	156	3.2 s	−20%
qpeexact	1019	175	2.3 s	87	2.2 s	−50%
vqe_su2	105	9	0.2 s	9	0.1 s	0%
wstate	70	6	0.1 s	8	0.0 s	+33%
gmean		**16.0**		**13.8**		**−14%**

TABLE VIII
SHARED KAHYPAR MAPPING: 64Q SUITE, H-GRID 2×3 4×4. DSABRE USES SABRE-STYLE FWD→BWD→FWD PASSES FROM PYTKET-DQC'S BEST-SEED PARTITION.

		pytket-dqc		DSABRE		
Circuit	CX	e-bits	time	EPR	time	Δ
ae	1962	519	7.9 s	226	11.2 s	−56%
ghz	63	7	0.1 s	7	0.0 s	0%
graphstate	64	9	0.2 s	8	0.0 s	−11%
qft	1966	591	9.6 s	229	9.2 s	−61%
qnn	8126	736	131.6 s	518	134.9 s	−30%
random	1627	1181	6.4 s	757	16.5 s	−36%
gmean		**160.0**		**102.2**		**−36%**

Discussion. Starting from an identical qubit-to-core partition, DSABRE reduces EPR consumption over pytket-dqc by a geometric mean of 55% on the 25q suite, 14% on the 36q suite, and 36% on the 64q suite. Bear in mind the counting asymmetry noted in the main paper: `pytket-dqc` amortises consecutive non-local gates into a single hyperedge, so its e-bit advantage is most pronounced on circuits with long runs of gates on the same cross-core wire. DSABRE's EPR advantage is largest for circuits with high CX density (ae, qft, qnn, random), where state teleportation moves a qubit once and all subsequent gates run locally regardless of their number. On the 36q suite the gmean advantage is narrower because `dj` regresses ($3 \to 7$ EPRs, +133%): its interaction graph is sparse enough that KaHyPar's static partition already achieves near-zero cross-core communication, and DSABRE's

dynamic teleport decisions then disrupt this near-optimal static assignment.

Compile time. pytket-dqc's wall time is dominated by KaHyPar and scales with the number of hyperedges (2-qubit gates): from $\sim$0.1 s on tiny circuits to 132 s on qnn-64q (5 seeds). DSABRE's routing time from a fixed initial layout is comparable for large circuits (135 s for qnn-64q) and negligible ($<$ 0.1 s) for small ones. For circuits with few 2-qubit gates, pytket-dqc's KaHyPar overhead exceeds DSABRE's routing time by one to two orders of magnitude.

APPENDIX E
COMPARISON WITH DMAPS

Background. DMapS [2] is a recent end-to-end DQC compiler that shares DSABRE's problem framing – jointly minimising EPR pair usage and local SWAP insertions on a heterogeneous quantum chip network – but takes a different architectural approach. Its mapping stage *DMapS-M* is two-stage: a KaHyPar hypergraph partition assigns logical qubits to chips, then a SABRE-style routine maps qubits to physical slots in parallel within each chip. Its routing stage *DMapS-R* is a SABRE-derived heuristic that, at every front-layer step, classifies non-executable two-qubit gates into four categories (local-only G_l, remote needing telegate G_{rnt}, remote-or-local G_{rao}, and remote-any G_{ran}) and *always prefers local SWAP candidates over remote ones* when both could satisfy a front-layer gate.

Cross-chip primitive in the output. Inspection of `router/car_layout_routing.py` shows that DMapS-R only ever inserts `SwapGate` ops – there is no Cat-Comm or telegate primitive emitted in the routed circuit. The coupling map handed to the router includes the inter-chip "remote connection" edges, so a front-layer CX whose two qubits happen to land on an inter-chip-link endpoint pair fires as a plain CX (cost-modelled as one EPR via Cat-Comm in `_rm_calculate_cost`); every other unexecutable two-qubit gate is resolved by inserting one or more SWAPs, of which the *remote* ones are the algorithm's main cross-chip mechanism. A remote SWAP that exchanges two qubits across a chip boundary can be decomposed into two one-way state teleportations – one for each direction – and therefore consumes *two* EPRs per remote SWAP. DSABRE's teledata, in contrast, performs only the one-way half: the incoming qubit lands in a slot that the layout has explicitly kept free, so no return teleport is needed and the cost is one EPR per relocation. This 2$\times$ asymmetry in per-relocation cost is the architectural source of the EPR gap observed below.

Implementation notes. We installed DMapS at commit `main` from https://github.com/RoccoLoter/DMapS in an isolated Python 3.10 conda environment, using the pinned `qiskit==0.39.2`, `kahypar==1.3.7`, and `retworkx` that DMapS requires. Five upstream changes were required to run DMapS over the full 18-circuit suite without crashes. None of them touch any algorithmic component of DMapS; they fix import-time, register-bookkeeping, and partition-bookkeeping bugs.

1) *Optional pytket-dqc baseline.* The top of `router/multi_mode_routing.py` unconditionally imports `tket_dqc_map` from DMapS's bundled `comparison_algorithms/pytket_dqc` subtree, which in turn needs a pytket-qiskit ABI that conflicts with DMapS's own qiskit pin. We wrapped the import in `try/except` and stubbed the symbols so that the DMapS-M+DMapS-R pipeline, which does not call them, still loads. The `tket_dqc` comparison baselines inside DMapS are not exercised here – they are not on our path and are evaluated separately in Appendix D.
2) *Hard-coded canonical-register size.* The output-conversion helper `_conv_circ_qubits_idx` in the same file allocates a final qiskit `QuantumRegister` with `size=66`. The author left a TODO comment to fix this. For any device with more than 66 physical qubits (in our case the $6\times16 = 96$-qubit H-grid), the routed circuit cannot be materialised. We replace the constant with `size = max(int(name[1:]) for name in qubits_name_n_idx) + 1`, derived from the device qubit-name map that DMapS already passes into the function.
3) *Multi-register circuit rewriting in `generate_new_circuit`.* The same file builds a single canonical register and rewrites every input gate as `Qubit(register=canonical_register, index=qubit.index)`, where `qubit.index` is the qubit's position *within its own original register*, not its global position. When the input circuit carries more than one quantum register – as the MQTBench-emitted QAE (`eval+q`) and QPE (`q+psi`) circuits do – the index-0 qubits of each original register both rewrite to `canonical[0]`. Any two-qubit gate touching both registers' index-0 qubits then triggers a `CircuitError('duplicate qubit arguments')`. We replaced `qubit.index` with the qubit's global circuit index obtained by `input_circ.find_bit(qubit).index`, eliminating the collision for any number of input registers. This fixes the failures on `ae-25`, `ae-64`, and `qpeexact-36`.
4) *Partitioner indexing of isolated qubits.* In `partitioner/qucirc_partitioner.py`, the hypergraph builder records each circuit qubit in `vers_wgt_dict` only if that qubit appears in some two-qubit-gate block, then later the `run()` method iterates `quantum_circuit.qubits` and indexes the resulting `qubits_tmp_idx` bidict by each one (line 104). Qubits that participate only in single-qubit gates – typical for the Bernstein–Vazirani query qubits in `bv-36`, where 18 of 36 qubits are idle under CX – never get a temp index and trip a `KeyError`. We pre-seed `vers_wgt_dict` with weight 1 for every qubit that appears in any op of the DAG (single-qubit ops included), but *not* for canonical-register slots that no op references; the latter are padding qubits introduced

when the device has more physical qubits than the input circuit uses, and giving them positive weight would inflate the initial-partition capacity demand and trip an `IndexError` in the block-filling loop. Weight is 1 (not 0) because KaHyPar 1.3.7 rejects zero-weight vertices. In the dependent comprehension at line 104 we additionally guard against padding qubits with `if qubit in qubits_tmp_idx`, which is safe because the resulting `init_partition` array is only used by a commented-out `setInitialPartition` call. This fixes the failure on `bv-36`.

5) *Multiprocessing entry-point.* `mapper/two_phase_mapper` uses `multiprocessing.Pool` for the per-chip parallel SABRE passes. On macOS the default start method is `spawn`, which re-imports the calling module; without an `if __name__ == "__main__"` guard a fork-bomb ensues. We added the guard to our driver script.

Patches (1)–(4) live inside DMapS and total fewer than thirty lines across two files; (5) is in our adapter only. All of them are marked with a `PATCH:` comment so they can be located by `grep` when DMapS is upgraded. With these in place, every circuit in the 18-circuit benchmark suite produces a routed circuit and a metrics record; the failure column of Table IX is now empty.

Driver script. A self-contained driver, `code/run_dmaps_bench.py`, performs the following for every (suite, circuit) pair:

1) Build the architecture from the same `build_b_grid_architecture(2,2,4)` or `build_h_grid_architecture(2,3,4)` call used by the main DSABRE benchmark, and serialise the chip graph plus inter-core links into DMapS's device JSON schema (`"has multiple chips"`, `"chips"`, `"remote connection"`). Physical qubits are renumbered so that DMapS's internal global index agrees with our chip-major ordering, giving the cheap `chip = idx // 16` lookup used later.
2) Run DMapS with default parameters (`chip_type="zcz"`, `intra_chip_all2all=False`, `heuristic="lookahead"`) five times per circuit. DMapS exposes no random seed, so reruns are independent KaHyPar partitions.
3) Classify every two-qubit op in the returned mapped circuit by the chip affiliations of its endpoints. A cross-chip CX (one that fires on an inter-chip coupling edge) contributes one EPR – the Cat-Comm price implicit in DMapS's cost model. A cross-chip SWAP contributes two EPRs, since it is realisable as two one-way teleportations exchanging the two qubits across the boundary. An intra-chip SWAP contributes one local SWAP. The overall overhead reported in Table IX uses the DMapS paper's weighting 1:10:20 (local SWAP : remote CX : remote SWAP).
4) Suppress KaHyPar's C-level stdout via file-descriptor redirection. Best-of-five and the full per-seed record are written to `code/results/results_dmaps_bench.json`.

Detailed results. Table IX reports best, mean, and standard deviation of overall overhead across five seeds for each circuit. With patches (3) and (4) above in place every circuit succeeds; the four originally-failing rows are marked with a † to indicate they only run after our partitioner / multi-register-rewrite fix. Without the patches, `ae` on the 25q and 64q suites and `qpeexact` on the 36q suite raise `CircuitError('duplicate qubit arguments')` from inside `generate_new_circuit` – the multi-register collision described in patch (3) – and `bv` on the 36q suite trips a `KeyError` in `partitioner/qucirc_partitioner.py:104` when iterating its 18 isolated query qubits. Neither failure is caused by our adapter; they reproduce on DMapS's own `tests/usage.ipynb` when one substitutes a multi-register or sparse-CX circuit.

Head-to-head with DSABRE. Table X reports the per-circuit EPR and local-SWAP counts side-by-side with the DSABRE variant of DSABRE (the one used in our headline tables). The ratio columns (EPR× and lSW×) are DMapS/DSABRE; values below 1 indicate DMapS wins on that metric.

Discussion. Two patterns are visible in Table X. First, on the four small structured circuits with very few cross-chip dependencies (`ghz`, `graphstate`, `wstate`, `vqe_su2`) DMapS matches or *beats* DSABRE on EPR – ratios in $\{0.19, 0.21, 0.50, 0.67, 1.00, 1.00\}$ – because it can occasionally fire a cross-chip CX directly across an inter-chip link (one EPR via Cat-Comm) without paying for any qubit relocation, whereas DSABRE always teleports a qubit first. Second, on every remaining circuit, including all of `qft`, `qnn`, `random`, `qaoa-36`, `ae`, `qpeexact-36`, `bv-36`, and `dj-36`, DMapS loses on EPR by between $1.59\times$ and $9.33\times$. The largest gaps are on circuits where DSABRE relocates a single hot qubit with one teleport and amortises many subsequent two-qubit gates locally (`bv-36`: DSABRE pays 1 EPR to move the answer bit once; DMapS pays 7 by firing five Cat-Comm CXs and one TP-Comm SWAP).

The breakdown in the `rCX` / `rSWAP` columns explains why. On the dense rows, the EPR spend that goes through remote SWAPs ($2\cdot$rSWAP) is comparable to or larger than the spend that goes through cross-chip CXs. For example `qnn-64q` attributes $\frac{2\cdot 280}{1013} \approx 55\%$ of its EPRs to remote SWAPs and `random-64q` attributes $\frac{2\cdot 370}{1053} \approx 70\%$. Each of those SWAPs is the two-teleport bidirectional exchange described above; DSABRE relocates the same qubit with a single one-way teleportation into a reserved free slot, paying half the EPR cost.

Compounding this, DMapS-M partitions the logical qubits with KaHyPar *before* routing and never relocates a qubit to a different chip mid-execution; the only way to move a qubit across chips at routing time is the same 2-EPR remote SWAP.

TABLE IX

DMAPS RESULTS ON OUR B-GRID (25Q, 36Q) AND H-GRID (64Q) SUITES, AFTER THE UPSTREAM PATCHES (3) AND (4) OF THE IMPLEMENTATION NOTES ARE APPLIED. BEST OF FIVE INDEPENDENT KAHYPAR PARTITIONS. EPR IS REMOTE-CX+2·REMOTE-SWAP. "OVERALL" USES DMAPS'S OWN 1:10:20 WEIGHTING (LOCAL SWAP : REMOTE CX : REMOTE SWAP). TIME IS CUMULATIVE WALL-CLOCK ACROSS ALL FIVE SEEDS. "MEAN±STDEV" SUMMARISES "OVERALL" ACROSS SEEDS. ROWS MARKED WITH † ORIGINALLY FAILED AND ONLY RUN AFTER THE CORRESPONDING PATCH.

Suite	Circuit	EPR	lSWAP	rCX	rSWAP	best	mean ± stdev	range	time
25q	ghz	1	4	1	0	14	15.6 ± 1.5	14–18	4.7 s
25q	graphstate	2	5	2	0	25	29.6 ± 5.2	25–38	4.9 s
25q	qft	121	231	49	36	1441	1620.0 ± 123.5	1441–1764	16.7 s
25q	qnn	121	504	51	35	1714	1783.8 ± 82.7	1714–1924	26.5 s
25q	random	342	1119	112	115	4539	4682.6 ± 131.8	4539–4846	56.7 s
25q	ae†	80	223	40	20	1023	1199.8 ± 130.0	1023–1317	16.2 s
36q	bv†	7	23	5	1	93	94.8 ± 1.3	93–96	5.1 s
36q	dj	28	115	18	5	395	429.2 ± 28.0	395–465	6.2 s
36q	qaoa	466	958	166	150	5618	6742.6 ± 1053.1	5618–8277	62.6 s
36q	qpeexact†	182	446	76	53	2266	2485.0 ± 194.2	2266–2760	26.9 s
36q	vqe_su2	6	8	6	0	68	72.0 ± 4.3	68–78	6.8 s
36q	wstate	4	2	4	0	42	46.0 ± 4.8	42–53	6.1 s
64q	ghz	3	17	3	0	47	50.4 ± 2.4	47–53	8.7 s
64q	graphstate	4	24	4	0	64	66.8 ± 3.1	64–72	8.9 s
64q	qft	392	1152	148	122	5072	5350.6 ± 247.4	5072–5648	74.2 s
64q	qnn	933	3762	405	264	13092	15422.6 ± 1666.4	13092–17035	306.2 s
64q	random	1138	3446	378	380	14826	15132.8 ± 278.4	14826–15555	150.9 s
64q	ae†	346	1119	116	115	4579	5167.8 ± 734.2	4579–6359	67.1 s

DSABRE's teleport action is available throughout routing, so it can correct mapping mistakes on the fly at one EPR each. The combination – $2\times$ per-relocation cost, plus an inability to amortise multiple remote gates on the same control qubit through a single relocation – is why the gap widens with circuit density rather than circuit size *per se*.

The local-SWAP picture is the dual of the EPR picture. Local SWAP ratios are below 1 for 14 of the 18 circuits (range 0.03 – 0.90); the four exceptions are the very-sparse `bv-36` and `dj-36` (ratios 2.88 and 4.79), where DSABRE routes essentially everything with a single teleport and a handful of local SWAPs, leaving DMapS's chain of remote SWAPs no room to save anything in absolute terms. On dense circuits, DMapS-R's explicit local-SWAP-first heuristic and its willingness to take the 2-EPR remote SWAP shortcut both reduce local SWAP count compared with DSABRE, which spends a small number of local SWAPs walking a qubit to a teleport launchpad before relocating it. The reduction in local SWAPs is not a saving in any meaningful sense: under the DMapS paper's own 1:10:20 weighting, the EPR overspend ($10\cdot\Delta\mathsf{rCX} + 20\cdot\Delta\mathsf{rSWAP}$) overwhelms the local-SWAP underspend on every dense row of Table X.

Variance behaviour confirms that the partition step dominates. Structured circuits have stdev/mean below 10% across the five seeds; dense circuits range 5 – 16%, with `qaoa-36q` reaching ± 1053 (range 5618 – 8277) and `qnn-64q` reaching ± 1666 (range 13092 – 17035). A single sub-optimal KaHyPar cut can change the cross-chip work by tens of percent. This is consistent with the original DMapS paper's own remark that partition quality bounds the achievable EPR. Cumulative wall-clock for the full 18-circuit run was approximately 15 minutes on a single Apple-silicon core.

Comm-qubit handling in the paper vs. in the released code. The DMapS paper draws a sharp distinction between *data qubits* (which store program information) and *communication qubits* (which exist only to create remote EPR pairs, with the device figures deliberately hiding the comm qubits). Each remote physical link, in the paper's text, "typically links one data qubit and two communication qubits" on either side, giving an intended topology of `data` → `comm` → `comm` → `data` across a chip boundary; the four-class gate taxonomy ($g_l, g_{rao}, g_{rnt}, g_{ran}$) refers explicitly to "data qubits connected to communication qubits" as the special boundary slots from which Cat-Comm CXs can be fired.

The released DMapS implementation does *not* carry this distinction. We verified two facts:

- All bundled device JSONs list every `"remote connection"` endpoint as a regular member of the corresponding chip's `qubits` array. For example, the file `tests/hardware_info/zcz/one_xiaohong_5module_ne` — the Net Zuchongzhi device used in the paper's own experiments — contains no separate comm-qubit entities. The qubits the loader flags `is_comm_qubit=True` are exactly those whose names appear in a `"remote connection"` entry; i.e. ordinary data qubits sitting on the chip boundary.
- `ScChip.qubits_num`, the value consumed as chip capacity by DMapS-M, is the full physical qubit list, comm flag-bearers included. The paper's remark that "their number is set equal to the data qubits adjacent to communication qubits" describes *intent* but is not enforced anywhere in the code.

TABLE X

EPR AND LOCAL-SWAP HEAD-TO-HEAD: DMAPS VS. DSABRE. DMAPS IS BEST OF FIVE KAHYPAR SEEDS (ITS OWN PROTOCOL); DSABRE IS BEST OF THREE SABRELAYOUT SEEDS, MATCHING THE MAIN-TEXT HEADLINE PROTOCOL. "RCX" AND "RSWAP" ARE THE CROSS-CHIP CX AND CROSS-CHIP SWAP COUNTS IN DMAPS'S OUTPUT CIRCUIT; DMAPS EPR $=$ RCX $+ 2\cdot$RSWAP. EPR$\times$ $=$ DMAPS EPR/DSABRE EPR; LSW$\times$ ANALOGOUSLY FOR LOCAL SWAPS. VALUES < 1 MEAN DMAPS USES FEWER EPRS (RESP. LOCAL SWAPS); VALUES > 1 MEAN DSABRE WINS. ROWS MARKED WITH † ARE CIRCUITS THAT ONLY RUN AFTER OUR UPSTREAM DMAPS PATCHES (3)/(4).

Suite	Circuit	CX	DSABRE EPR	DSABRE lSW	rCX	rSWAP	DMapS EPR	DMapS lSW	EPR$\times$	lSW$\times$
25q	ghz	24	1	14	1	0	1	4	1.00	0.29
25q	graphstate	25	2	16	2	0	2	5	1.00	0.31
25q	qft	580	33	461	49	36	121	231	3.67	0.50
25q	qnn	1223	48	1076	51	35	121	504	2.52	0.47
25q	random	1124	169	1374	112	115	342	1119	2.02	0.81
25q	ae†	558	23	374	40	20	80	223	3.48	0.60
36q	bv†	17	1	8	5	1	7	23	7.00	2.88
36q	dj	35	3	24	18	5	28	115	9.33	4.79
36q	qaoa	1200	145	1319	166	150	466	958	3.21	0.73
36q	qpeexact†	1019	65	874	76	53	182	446	2.80	0.51
36q	vqe_su2	105	9	52	6	0	6	8	0.67	0.15
36q	wstate	70	8	63	4	0	4	2	0.50	0.03
64q	ghz	63	16	89	3	0	3	17	0.19	0.19
64q	graphstate	64	19	104	4	0	4	24	0.21	0.23
64q	qft	1966	246	2054	148	122	392	1152	1.59	0.56
64q	qnn	8126	521	8662	405	264	933	3762	1.79	0.43
64q	random	1627	714	3814	378	380	1138	3446	1.59	0.90
64q	ae†	1962	216	2332	116	115	346	1119	1.60	0.48

Consequently, inter-chip links in the loaded device are modelled as a single coupling edge between two ordinary data qubits: the `data` $\rightarrow$ `comm` $\rightarrow$ `comm` $\rightarrow$ `data` four-hop path of the paper is collapsed to a single `data` $\rightarrow$ `data` edge. This matches the code-level audit in the previous subsection: every cross-chip CX or SWAP in the routed output lands on exactly one of these edges. The Cat-Comm and TP-Comm realisations of those cross-chip operations are not materialised in the routed circuit; they only enter through the `_rm_calculate_cost` scoring term that charges $2 \cdot$ *chip_dist* $- 1$ EPRs per cross-chip CX.

Our adapter inherits the same modelling: each inter-chip link is one edge between two of the chip's data qubits, those endpoints are flagged `is_comm_qubit`, and chip capacity is the full 16-qubit grid. This matches DSABRE's architecture (`code/architecture.py`), which also flags inter-core-link endpoints as comm qubits without removing them from the data-qubit pool, so the head-to-head comparison is fair: both compilers see the same number of usable slots per chip and the same inter-chip topology.

Reproducing the table.

```
# from the dsabre/ root
/opt/anaconda3/envs/dmaps/bin/python \
    code/run_dmaps_bench.py 25 36 64
# writes code/results/results_dmaps_bench.json
```

The JSON additionally records the full per-seed trace (`seeds: [...]`), so future runs can extract any metric without re-running the benchmark. Three artefacts together suffice to reproduce the numbers verbatim, modulo KaHyPar's internal randomness: the conda environment specification, the four upstream DMapS patches (1)–(4) listed under Implementation Notes, and the adapter implementation `code/run_dmaps_bench.py`.

Fill-ratio sweep. The discussion above implies that DMapS's relative EPR cost should shrink as the number of logical qubits approaches the number of physical qubits, because DSABRE's one-way teleport advantage depends on the existence of a free physical slot for the incoming qubit to land in. In the limit of zero free slots every relocation has to displace an occupant, and dSABRE's cheapest move becomes a full 2-EPR exchange – the same cost as DMapS's remote SWAP. To test this directly we ran the QFT family at $n \in \{24, 36, 48, 56\}$ logical qubits on the same 64-qubit B-grid, varying *fill* ($n/64 \in \{37.5\%, 56.2\%, 75.0\%, 87.5\%\}$) while keeping the architecture fixed. Each row is best of five seeds, generated from `code/run_fill_sweep.py`.

TABLE XI

FILL-RATIO SWEEP ON A FIXED 4-CHIP B-GRID (64 PHYS). SAME QFT FAMILY DECOMPOSED TO {SX, RZ, CX, X} AT FOUR INPUT SIZES. EPR$\times$ $=$ DMAPS EPR/DSABRE EPR; LSW$\times$ ANALOGOUSLY FOR LOCAL SWAPS. "FILL" IS $n/64$.

			DSABRE		DMapS		ratio	
n	fill	CX	EPR	lSW	EPR	lSW	EPR$\times$	lSW$\times$
24	37.5%	588	30	446	118	238	3.93	0.53
36	56.2%	1314	88	1153	406	803	4.61	0.70
48	75.0%	2272	159	2084	518	1301	3.26	0.62
56	87.5%	2924	819	4423	831	1523	**1.01**	**0.34**

The EPR$\times$ column drops from ≈ 4 at low fill to *essentially parity* (1.01) at 87.5% fill, matching the prediction. The mechanism is visible in DSABRE's own numbers: between $n = 48$ and $n = 56$ DSABRE's CX count grows by only a factor 1.29 but its EPR count grows by a factor 5.2 – the free-slot pool collapses and the router is forced into slot-eviction churn, doubling the effective per-relocation cost. DMapS's EPR grows by only $1.6\times$ over the same step because its remote-SWAP-only relocation is insensitive to fill ratio by construction.

None of the headline benchmark rows reaches the 87.5% fill regime: the 25q suite occupies 39% of the B-grid, the 36q suite 56%, and the 64q suite 67% of the H-grid. The fill sweep therefore documents a regime in which the head-to-head numbers no longer favour DSABRE. It also explains why DSABRE dominates the lower-fill regimes that are typical of the near-term DQC setups targeted in this paper.

Reproducing the fill sweep. The DSABRE side runs in the main repo's qiskit 2.x environment, the DMapS side in the isolated DMapS conda environment:

```
# dSABRE side (main repo env)
python code/run_fill_sweep.py dsabre

# DMapS side (isolated env)
/opt/anaconda3/envs/dmaps/bin/python \
    code/run_fill_sweep.py dmaps
```

REFERENCES

[1] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[2] T. Luo, Y. Zheng, Y. Deng, and X. Fu, "DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: https://github.com/RoccoLoter/DMapS.